AG3 - Actividad Guiada 3

Nombre: Karen Nattaly Valero González
Link: https://colab.research.google.com/drive/1a2974oxYBlGwPcO2ltsaO7tR2CI7nBtT?usp=sharing
Github: https://github.com/knvalerog22/Algoritmos_optimizacion/AG3

Carga de librerias

```
!pip install requests
!pip install tabulate>=0.9 networkx>=3.0
!pip install tsplib95 --no-deps
!pip install deprecated

```

```
Requirement already satisfied: requests in /usr/local/lib/python3.12/dist-packages (2.32.4)
Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (from requests) (3.4.7)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages (from requests) (3.11)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages (from requests) (2.5.0)
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/dist-packages (from requests) (2026.2.25)
Collecting tsplib95
  Downloading tsplib95-0.7.1-py2.py3-none-any.whl.metadata (6.3 kB)
Downloading tsplib95-0.7.1-py2.py3-none-any.whl (25 kB)
Installing collected packages: tsplib95
Successfully installed tsplib95-0.7.1
Collecting deprecated
  Downloading deprecated-1.3.1-py2.py3-none-any.whl.metadata (5.9 kB)
Requirement already satisfied: wrapt<3,>=1.10 in /usr/local/lib/python3.12/dist-packages (from deprecated) (2.1.2)
Downloading deprecated-1.3.1-py2.py3-none-any.whl (11 kB)
Installing collected packages: deprecated
ERROR: pip's dependency resolver does not currently take into account all the packages that are installed. This behaviour is the source of the followi
tsplib95 0.7.1 requires Deprecated~=1.2.9, but you have deprecated 1.3.1 which is incompatible.
tsplib95 0.7.1 requires networkx~=2.1, but you have networkx 3.6.1 which is incompatible.
tsplib95 0.7.1 requires tabulate~=0.8.7, but you have tabulate 0.9.0 which is incompatible.
Successfully installed deprecated-1.3.1
```

Carga de los datos del problema

```

import urllib.request #Hacer llamadas http a paginas de la red
import tsplib95       #Modulo para las instancias del problema del TSP
import math           #Modulo de funciones matematicas. Se usa para exp
import random         #Para generar valores aleatorios


#Descargamos el fichero de datos(Matriz de distancias)
file = "swiss42.tsp" ;
#urllib.request.urlretrieve("https://raw.githubusercontent.com/mastqe/tsplib/refs/heads/master/swiss42.tsp", file + '.gz')
urllib.request.urlretrieve("https://raw.githubusercontent.com/mastqe/tsplib/refs/heads/master/swiss42.tsp", file  )
#!gzip -d swiss42.tsp.gz     #Descomprimir el fichero de datos

#Coordendas 51-city problem (Christofides/Eilon)
#file = "eil51.tsp" ; urllib.request.urlretrieve("http://comopt.ifi.uni-heidelberg.de/software/TSPLIB95/tsp/eil51.tsp.gz", file)

#Coordenadas - 48 capitals of the US (Padberg/Rinaldi)
#file = "att48.tsp" ; urllib.request.urlretrieve("http://comopt.ifi.uni-heidelberg.de/software/TSPLIB95/tsp/att48.tsp.gz", file)


#http://comopt.ifi.uni-heidelberg.de/software/TSPLIB95
#Documentacion :
  # http://comopt.ifi.uni-heidelberg.de/software/TSPLIB95/tsp95.pdf
  # https://tsplib95.readthedocs.io/en/stable/pages/usage.html
  # https://tsplib95.readthedocs.io/en/v0.6.1/modules.html
  # https://pypi.org/project/tsplib95/


```

```
('swiss42.tsp', <http.client.HTTPMessage at 0x7a55243b2ff0>)
```

Carga de datos y generación de objeto problema

```
problem = tsplib95.load(file)

#Nodos
Nodos = list(problem.get_nodes())

#Aristas
Aristas = list(problem.get_edges())

print(problem)

```

```
NAME: swiss42
COMMENT: 42 Staedte Schweiz (Fricker)
TYPE: TSP
DIMENSION: 42
EDGE_WEIGHT_TYPE: EXPLICIT
```

```
EDGE_WEIGHT_FORMAT: FULL_MATRIX
EDGE_WEIGHT_SECTION:
0 15 30 23 32 55 33 37 92 114 92 110 96 90 74 76 82 67 72 78 82 159 122 131 206 112 57 28 43 70 65 66 37 103 84 125 129 72 126 141 183 124
15 0 34 23 27 40 19 32 93 117 88 100 87 75 63 67 71 69 62 63 96 164 132 131 212 106 44 33 51 77 75 72 52 118 99 132 132 67 139 148 186 122
30 34 0 11 18 57 36 65 62 84 64 89 76 93 95 100 104 98 57 88 99 130 100 101 179 86 51 4 18 43 45 95 45 115 93 152 159 100 112 114 153 94
23 23 11 0 11 48 26 54 70 94 69 89 75 84 84 89 92 89 54 78 99 141 111 109 190 89 44 11 29 54 56 89 47 118 96 147 151 90 122 126 163 101
32 27 18 11 0 40 20 58 67 92 61 78 65 76 83 89 91 95 43 72 110 141 116 105 190 81 34 19 35 57 63 97 58 129 107 156 158 92 129 127 161 95
55 40 57 48 40 0 23 55 96 123 78 75 62 36 56 66 63 95 37 34 137 174 156 129 224 90 15 59 75 96 103 105 91 158 139 164 156 78 169 163 191 115
33 19 36 26 20 23 0 45 85 111 75 82 69 60 63 70 71 85 44 52 115 161 136 122 210 91 25 37 54 78 81 90 68 136 116 150 147 76 148 147 180 111
37 32 65 54 58 55 45 0 124 149 118 126 113 80 42 42 49 40 87 60 94 195 158 163 242 135 65 63 79 106 101 50 66 118 104 109 103 36 160 178 218 153
92 93 62 70 67 96 85 124 0 28 29 68 63 122 148 155 156 159 67 129 148 78 80 39 129 46 82 65 55 40 61 157 97 159 135 212 221 159 110 72 95 35
114 117 84 94 92 123 111 149 28 0 54 91 88 150 174 181 182 181 95 157 159 50 65 27 102 65 110 87 73 50 68 176 112 166 142 229 241 184 99 46 69 38
92 88 64 69 61 78 75 118 29 54 0 39 34 99 134 142 141 157 44 110 161 103 109 52 154 22 63 68 66 61 81 158 107 175 151 216 219 150 137 100 115 37
110 100 89 89 78 75 82 126 68 91 39 0 14 80 129 139 135 167 39 98 187 136 148 81 186 28 61 92 97 98 117 173 134 204 181 232 229 153 176 137 143 62
96 87 76 75 65 62 69 113 63 88 34 14 0 72 117 128 124 153 26 88 174 136 142 82 187 32 48 79 85 89 106 159 121 191 168 219 216 140 168 134 145 64
90 75 93 84 76 36 60 80 122 150 99 80 72 0 59 71 63 116 56 25 170 201 189 151 252 104 44 95 111 130 138 130 127 192 174 186 172 90 205 193 214 135
74 63 95 84 83 56 63 42 148 174 134 129 117 59 0 11 8 63 93 35 135 223 195 184 273 146 71 95 113 138 138 81 107 159 146 132 113 32 200 209 243 171
76 67 100 89 89 66 70 42 155 181 142 139 128 71 11 0 11 54 103 46 130 230 198 192 279 155 80 99 117 143 141 74 107 155 143 122 102 22 202 215 250 179
82 71 104 92 91 63 71 49 156 182 141 135 124 63 8 11 0 65 100 39 140 232 203 192 281 153 78 103 121 147 146 85 115 164 152 133 112 33 208 218 251 178
67 69 98 89 95 95 85 40 159 181 157 167 153 116 63 54 65 0 127 92 83 224 180 199 269 175 106 95 109 135 125 21 80 107 100 71 63 33 173 205 249 191
72 62 57 54 43 37 44 87 67 95 44 39 26 56 93 103 100 127 0 67 153 145 139 96 196 53 23 60 70 81 95 134 101 172 149 194 190 115 160 138 159 80
78 63 88 78 72 34 52 60 129 157 110 98 88 25 35 46 39 92 67 0 152 207 188 162 258 119 48 89 107 129 134 108 114 176 159 163 147 66 200 197 224 147
82 96 99 99 110 137 115 94 148 159 161 187 174 170 135 130 140 83 153 152 0 188 128 184 222 183 139 95 95 110 91 62 54 24 23 81 110 113 108 164 217 18
159 164 130 141 141 174 161 195 78 50 103 136 136 201 223 230 232 224 145 207 188 0 65 57 51 109 160 132 116 90 102 217 148 188 168 264 281 231 100 26
122 132 100 111 116 156 136 158 80 65 109 148 142 189 195 198 203 180 139 188 128 65 0 91 94 126 145 100 82 60 57 167 99 126 106 208 230 194 36 39 94
131 131 101 109 105 129 122 163 39 27 52 81 82 151 184 192 192 199 96 162 184 57 91 0 106 53 115 104 94 74 94 196 134 192 168 251 260 197 126 64 64 19
206 212 179 190 190 224 210 242 129 102 154 186 187 252 273 279 281 269 196 258 222 51 94 106 0 158 211 180 163 136 145 259 190 218 200 302 323 278 12
112 106 86 89 81 90 91 135 46 65 22 28 32 104 146 155 153 175 53 119 183 109 126 53 158 0 75 89 88 83 103 178 129 197 173 236 238 166 156 111 115 34
57 44 51 44 34 15 25 65 82 110 63 61 48 44 71 80 78 106 23 48 139 160 145 115 211 75 0 53 68 86 95 114 90 160 139 173 168 92 162 150 176 101
28 33 4 11 19 59 37 63 65 87 68 92 79 95 95 99 103 95 60 89 95 132 100 104 180 89 53 0 18 44 45 92 42 112 89 149 156 99 111 116 155 97
43 51 18 29 35 75 54 79 55 73 66 97 85 111 113 117 121 109 70 107 95 116 82 94 163 88 68 18 0 27 27 103 42 109 85 157 168 115 94 98 140 90
70 77 43 54 57 96 78 106 40 50 61 98 89 130 138 143 147 135 81 129 110 90 60 74 136 83 86 44 27 0 21 128 62 119 96 179 192 142 79 72 115 74
65 75 45 56 63 103 81 101 61 68 81 117 106 138 138 141 146 125 95 134 91 102 57 94 145 103 95 45 27 21 0 115 46 98 75 163 179 136 67 81 129 95
66 72 95 89 97 105 90 50 157 176 158 173 159 130 81 74 85 21 134 108 62 217 167 196 259 178 114 92 103 128 115 0 69 86 81 60 65 54 158 195 243 190
37 52 45 47 58 91 68 66 97 112 107 134 121 127 107 107 115 80 101 114 54 148 99 134 190 129 90 42 42 62 46 69 0 71 49 117 133 98 95 127 175 132
103 118 115 118 129 158 136 118 159 166 175 204 191 192 159 155 164 107 172 176 24 188 126 192 218 197 160 112 109 119 98 86 71 0 24 94 127 137 100 16
84 99 93 96 107 139 116 104 135 142 151 181 168 174 146 143 152 100 149 159 23 168 106 168 200 173 139 89 85 96 75 81 49 24 0 104 133 127 85 143 197 1
125 132 152 147 156 164 150 109 212 229 216 232 219 186 132 122 133 71 194 163 81 264 208 251 302 236 173 149 157 179 163 60 117 94 104 0 39 100 190 2
129 132 159 151 158 156 147 103 221 241 219 229 216 172 113 102 112 63 190 147 110 281 230 260 323 238 168 156 168 192 179 65 133 127 133 39 0 81 216
72 67 100 90 92 78 76 36 159 184 150 153 140 90 32 22 33 33 115 66 113 231 194 197 278 166 92 99 115 142 136 54 98 137 127 100 81 0 193 214 253 187
126 139 112 122 129 169 148 160 110 99 137 176 168 205 200 202 208 173 160 200 108 100 36 126 120 156 162 111 94 79 67 158 95 100 85 190 216 193 0 74
141 148 114 126 127 163 147 178 72 46 100 137 134 193 209 215 218 205 138 197 164 26 39 64 65 111 150 116 98 72 81 195 127 163 143 241 259 214 74 0 55
183 186 153 163 161 191 180 218 95 69 115 143 145 214 243 250 251 249 159 224 217 30 94 64 49 115 176 155 140 115 129 243 175 218 197 292 307 253 129
124 122 94 101 95 115 111 153 35 38 37 62 64 135 171 179 178 191 80 147 184 75 103 19 124 34 101 97 90 74 95 190 132 194 170 246 253 187 137 80 81 0
EOF
```

```
print(Nodos)
```

```
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35, 36, 37, 38, 39,
```

```
print(Aristas)
# https://github.com/mastqe/tsplib/blob/master/swiss42.tsp
```

```
[(0, 0), (0, 1), (0, 2), (0, 3), (0, 4), (0, 5), (0, 6), (0, 7), (0, 8), (0, 9), (0, 10), (0, 11), (0, 12), (0, 13), (0, 14), (0, 15), (0, 16), (0, 17
```

```
NOMBRE: swiss42
TIPO: TSP
COMENTARIO: 42 Staedte Schweiz (Fricker)
DIMENSION: 42
EDGE_WEIGHT_TYPE: EXPLICIT
EDGE_WEIGHT_FORMAT: FULL_MATRIX
EDGE_WEIGHT_SECTION
0 15 30 23 32 55 33 37 92 114 92 110 96 90 74 76 82 72 78 82 159 122 131 206 112 57 28 43 70
  15 0 34 23 27 40 19 32 93 117 88 100 87 75 63 67 71 69 62 63 96 164 132 131 212 106 44 33 5
  30 34 0 11 18 57 36 65 62 84 64 89 76 93 95 100 104 98 57 88 99 130 100 101 179 86 51 4 18
  23 23 11 0 11 48 26 54 70 94 69 75 75 84 84 89 92 89 54 78 99 141 111 109 89 89 11 11 54
  32 27 18 11 0 40 20 58 67 92 61 78 65 76 83 89 91 95 43 72 110 141 116 105 190 81 34 19 35
  55 40 57 48 40 0 23 55 96 123 78 75 36 36 66 66 63 95 34 34 137 174 156 129 224 90 15 59 75
  33 19 36 26 20 23 0 45 85 111 75 82 69 60 63 70 71 85 44 52 115 161 136 122 210 91 25 37 54
  37 32 65 54 58 55 45 0 124 149 118 126 113 80 42 42 40 40 87 87 94 158 158 163 242 135 65 6
  92 93 62 70 67 96 85 124 0 28 29 68 63 122 148 155 156 159 67 129 148 78 80 39 129 46 82 65
 114 117 84 94 92 123 111 149 28 0 54 91 88 150 174 181 182 181 95 157 159 50 65 27 102 65 11
  92 88 64 69 61 78 75 118 29 54 0 39 34 99 134 142 141 157 44 110 161 103 109 52 154 22 63 6
 110 100 89 89 78 75 82 126 68 91 39 0 14 80 129 139 135 167 39 98 187 136 148 81 186 28 61 9
  96 87 76 75 65 62 69 113 63 88 34 14 0 72 117 128 124 153 26 88 174 136 142 82 187 32 48 79
  90 75 93 84 76 36 60 80 122 150 99 80 72 0 59 71 63 116 56 25 170 201 189 151 252 104 44 95
  74 63 95 84 83 56 63 42 148 174 134 129 117 59 0 11 8 63 93 35 135 223 195 184 273 146 71 9
```

```
#Probamos algunas funciones del objeto problem

#Distancia entre nodos
problem.get_weight(3, 0)

#Todas las funciones
#Documentación: https://tsplib95.readthedocs.io/en/v0.6.1/modules.html

#dir(problem)
```

```
23
```

Funcionas basicas

```
#Se genera una solucion aleatoria con comienzo en en el nodo 0
def crear_solucion(Nodos):
  solucion = [Nodos[0]]
  for n in Nodos[1:]:
    solucion = solucion + [random.choice(list(set(Nodos) - set({Nodos[0]}) - set(solucion)))]
  return solucion
```

```python

#Devuelve la distancia entre dos nodos
def distancia(a,b, problem):
  return problem.get_weight(a,b)

#Devuelve la distancia total de una trayectoria/solucion
def distancia_total(solucion, problem):
  distancia_total = 0
  for i in range(len(solucion)-1):
    distancia_total += distancia(solucion[i] ,solucion[i+1] ,  problem)
  return distancia_total + distancia(solucion[len(solucion)-1] ,solucion[0], problem)

sol_temporal = crear_solucion(Nodos)

print(len(sol_temporal))
print(sol_temporal   )
print(distancia_total(sol_temporal, problem))
```

```
42
[0, 6, 21, 8, 11, 10, 37, 1, 32, 34, 16, 7, 35, 18, 30, 24, 33, 31, 9, 20, 5, 3, 38, 39, 15, 13, 17, 27, 28, 22, 12, 19, 23, 26, 41, 14, 29, 40, 25, 4
4633
```

BUSQUEDA ALEATORIA

```python
def busqueda_aleatoria(problem, N):
  #N es el numero de iteraciones
  Nodos = list(problem.get_nodes())

  mejor_solucion = []
  #mejor_distancia = 10e100                        #Inicializamos con un valor alto
  mejor_distancia = float('inf')                   #Inicializamos con un valor alto

  for i in range(N):                               #Criterio de parada: repetir N veces pero podemos incluir otros
    solucion = crear_solucion(Nodos)               #Genera una solucion aleatoria
    distancia = distancia_total(solucion, problem)  #Calcula el valor objetivo(distancia total)

    if distancia < mejor_distancia:                #Compara con la mejor obtenida hasta ahora
      mejor_solucion = solucion
      mejor_distancia = distancia


  print("Mejor solución:" , mejor_solucion)
  print("Distancia     :" , mejor_distancia)
  return mejor_solucion


#Busqueda aleatoria con 5000 iteraciones
solucion = busqueda_aleatoria(problem, 50000)
```

```
Mejor solución: [0, 28, 4, 39, 22, 24, 11, 12, 26, 15, 13, 18, 2, 5, 35, 33, 38, 3, 20, 21, 40, 25, 32, 8, 29, 1, 6, 16, 31, 7, 37, 36, 41, 9, 30, 23,
Distancia     : 3662
```

BÚSQUEDA LOCAL

```python
# BUSQUEDA LOCAL(1 paso)

def genera_vecina(solucion):
  #Generador de soluciones vecinas: 2-opt (intercambiar 2 nodos) Si hay N nodos se generan (N-1)x(N-2)/2 soluciones
  #Se puede modificar para aplicar otros generadores distintos que 2-opt
  #print(solucion)
  mejor_solucion = []
  mejor_distancia = 10e100
  for i in range(1,len(solucion)-1):          #Recorremos todos los nodos en bucle doble para evaluar todos los intercambios 2-opt
    for j in range(i+1, len(solucion)):

      #Se genera una nueva solución intercambiando los dos nodos i,j:
      #  (usamos el operador + que para listas en python las concatena) : ej.: [1,2] + [3] = [1,2,3]
      vecina = solucion[:i] + [solucion[j]] + solucion[i+1:j] + [solucion[i]] + solucion[j+1:]

      #Se evalua la nueva solución ...
      distancia_vecina = distancia_total(vecina, problem)

      #... para guardarla si mejora las anteriores
      if distancia_vecina <= mejor_distancia:
        mejor_distancia = distancia_vecina
        mejor_solucion = vecina
  return mejor_solucion


#solucion = [1, 47, 13, 41, 40, 19, 42, 44, 37, 5, 22, 28, 3, 2, 29, 21, 50, 34, 30, 9, 16, 11, 38, 49, 10, 39, 33, 45, 15, 24, 43, 26, 31, 36, 3
print("Distancia Solucion Incial:" , distancia_total(solucion, problem))


nueva_solucion = genera_vecina(solucion)
print("Distancia Mejor Solucion Local:", distancia_total(nueva_solucion, problem))

```

```
Distancia Solucion Incial: 3662
Distancia Mejor Solucion Local: 3409
```

```python
#Busqueda Local(iteraciones):
#  - Sobre el operador de vecindad 2-opt(funcion genera_vecina)
#  - Sin criterio de parada, se para cuando no es posible mejorar.
def busqueda_local(solucion, problem):
  mejor_solucion = []

  #Generar una solucion inicial de referencia(aleatoria)
  #solucion_referencia = crear_solucion(Nodos)
  solucion_referencia = solucion
  mejor_distancia = distancia_total(solucion_referencia, problem)

  iteracion=0             #Un contador para saber las iteraciones que hacemos
  while(1):
    iteracion +=1         #Incrementamos el contador
    #print('#',iteracion)

    #Obtenemos la mejor vecina ...
    vecina = genera_vecina(solucion_referencia)

    #... y la evaluamos para ver si mejoramos respecto a lo encontrado hasta el momento
    distancia_vecina = distancia_total(vecina, problem)

    #Si no mejoramos hay que terminar. Hemos llegado a un minimo local(según nuestro operador de vencindad 2-opt)
    if distancia_vecina < mejor_distancia:
      #mejor_solucion = copy.deepcopy(vecina)   #Con copia profunda. Las copias en python son por referencia
      mejor_solucion = vecina                   #Guarda la mejor solución encontrada
      mejor_distancia = distancia_vecina

    else:
      print("En la iteracion ", iteracion, ", la mejor solución encontrada es:" , mejor_solucion)
      print("Distancia     :" , mejor_distancia)
      return mejor_solucion

    solucion_referencia = vecina


sol = busqueda_local(nueva_solucion, problem )
```

```
En la iteracion  31 , la mejor solución encontrada es: [0, 4, 9, 23, 41, 25, 11, 12, 18, 26, 5, 13, 19, 36, 35, 20, 33, 34, 38, 24, 40, 21, 39, 22, 30
Distancia     : 1727
```

SIMULATED ANNEALING

```python
#Generador de 1 solucion vecina 2-opt 100% aleatoria (intercambiar 2 nodos)
#Mejorable eligiendo otra forma de elegir una vecina.
def genera_vecina_aleatorio(solucion):

  #Se eligen dos nodos aleatoriamente
  i,j = sorted(random.sample( range(1,len(solucion)) , 2))

  #Devuelve una nueva solución pero intercambiando los dos nodos elegidos al azar
  return solucion[:i] + [solucion[j]] + solucion[i+1:j] + [solucion[i]] + solucion[j+1:]


#Funcion de probabilidad para aceptar peores soluciones
def probabilidad(T,d):
  if random.random() <  math.exp( -1*d / T)  :
    return True
  else:
    return False

#Funcion de descenso de temperatura
def bajar_temperatura(T):
  return T*0.99
```

```python
def recocido_simulado(problem, TEMPERATURA ):
  #problem = datos del problema
  #T = Temperatura

  solucion_referencia = crear_solucion(Nodos)
  distancia_referencia = distancia_total(solucion_referencia, problem)

  mejor_solucion = []             #x* del seudocodigo
  mejor_distancia = 10e100        #F* del seudocodigo


  N=0
  while TEMPERATURA > .0001:
    N+=1
    #Genera una solución vecina
    vecina =genera_vecina_aleatorio(solucion_referencia)

    #Calcula su valor(distancia)
    distancia_vecina = distancia_total(vecina, problem)

    #Si es la mejor solución de todas se guarda(siempre!!!)
    if distancia_vecina < mejor_distancia:
        mejor_solucion = vecina
        mejor_distancia = distancia_vecina

    #Si la nueva vecina es mejor se cambia
    #Si es peor se cambia según una probabilidad que depende de T y delta(distancia_referencia - distancia_vecina)
    if distancia_vecina < distancia_referencia or probabilidad(TEMPERATURA, abs(distancia_referencia - distancia_vecina) ) :
```

```
      #solucion_referencia = copy.deepcopy(vecina)
      solucion_referencia = vecina
      distancia_referencia = distancia_vecina

    #Bajamos la temperatura
    TEMPERATURA = bajar_temperatura(TEMPERATURA)

  print("La mejor solución encontrada es " , end="")
  print(mejor_solucion)
  print("con una distancia total de " , end="")
  print(mejor_distancia)
  return mejor_solucion

sol  = recocido_simulado(problem, 10000000)
```

```
La mejor solución encontrada es [0, 27, 2, 12, 18, 26, 6, 16, 15, 1, 3, 30, 29, 10, 8, 9, 39, 22, 21, 40, 24, 23, 41, 25, 11, 13, 14, 37, 36, 35, 20,
con una distancia total de 1954
```

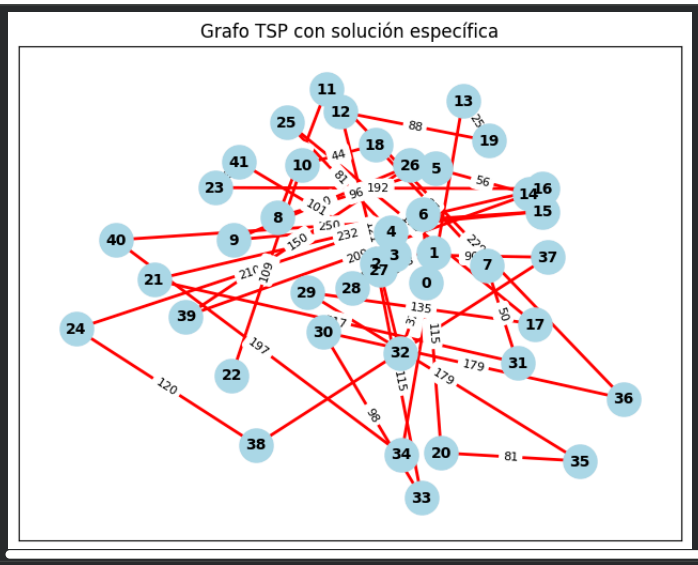
Representación en un grafo a partir de la matriz de distancias

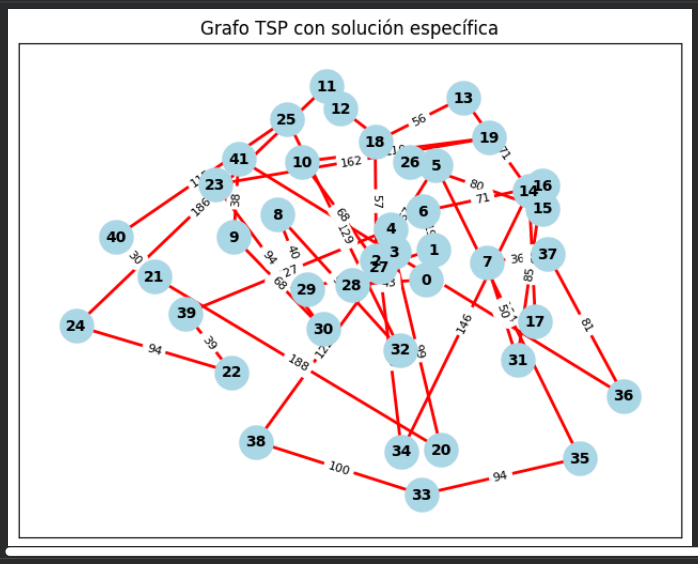
(Optimización de posiciones usando escalado multidimensional (MDS)

Multidimensional scaling problem(MDS): https://en.wikipedia.org/wiki/Multidimensional_scaling

```
import numpy as np
import networkx as nx
import matplotlib.pyplot as plt
from sklearn.manifold import MDS  # Multidimensional Scaling o Escalado Multidimensional

def plot_tsp_solution(distance_matrix, tsp_solution):
    """
    Dibuja el grafo de un TSP con las posiciones calculadas mediante MDS y muestra
    solo las aristas correspondientes a la solución del TSP.

    :param distance_matrix: np.ndarray, matriz de distancias entre nodos
    :param tsp_solution: list, lista de nodos en el orden de la solución del TSP
    """
    # Crear el grafo completo
    G = nx.Graph()
    num_nodes = len(distance_matrix)
    for i in range(num_nodes):
        for j in range(i + 1, num_nodes):
            G.add_edge(i, j, weight=distance_matrix[i][j])

    # Usar MDS para calcular posiciones de los nodos
    mds = MDS(n_components=2, dissimilarity="precomputed", random_state=42)
    positions = mds.fit_transform(distance_matrix)

    # Convertir las posiciones en un diccionario para networkx
    pos = {i: positions[i] for i in range(num_nodes)}

    # Crear un subgrafo con las aristas del camino TSP
    TSP_G = nx.Graph()
    for i in range(len(tsp_solution) - 1):
        u = tsp_solution[i]
        v = tsp_solution[i + 1]
        TSP_G.add_edge(u, v, weight=distance_matrix[u][v])

    # Dibujar el grafo
    plt.figure(figsize=(8, 6))

    # Dibujar nodos
    nx.draw_networkx_nodes(G, pos, node_color='lightblue', node_size=500)

    # Dibujar las aristas del camino TSP
    nx.draw_networkx_edges(TSP_G, pos, edge_color='red', width=2)

    # Añadir etiquetas a los nodos y pesos de las aristas
    nx.draw_networkx_labels(G, pos, font_size=10, font_weight='bold')
    edge_labels = nx.get_edge_attributes(TSP_G, 'weight')
    nx.draw_networkx_edge_labels(TSP_G, pos, edge_labels=edge_labels, font_size=8)

    plt.title("Grafo TSP con solución específica")
    plt.show()

```

```
plot_tsp_solution(problem.edge_weights, crear_solucion(Nodos))
```




```
plot_tsp_solution(problem.edge_weights, solucion)
```